

Project Documentation
Cyphal

Filip Radojević

February 20, 2026

Contents

1	Introduction	2
2	Cyphal Messages and Build System	3
2.1	Overview	3
2.2	MAVLink to DSDL Conversion	3
2.3	Official Cyphal Data Types	4
2.4	Automatic Code Generation	4
2.5	Figures: Memory Pools	5
3	Libcanard	7
3.1	Overview	7
3.2	Memory Architecture	7
3.3	Transmit Queue (TXQ)	8
3.4	Initialization Flow	9
3.5	Initialization of Cyphal Components	9
3.5.1	Instance Pool	9
3.5.2	Main Canard Instance	9
3.5.3	Transmit Queue (TXQ)	10
3.6	Subscription to messages	11
3.7	Processing Incoming Frames	13
3.8	Publishing Messages	16
3.9	Draining the Transmit Queue	17
4	Middleware for Cyphal Files	20
5	Overall Cyphal Communication Flow	21

Chapter 1

Introduction

The initial system was implemented using MAVLink messages organized in a star topology. While functional during early development, the increasing demand for reliable UDP message handling and the limitations of the star configuration led to reduced maintainability and scalability. Ensuring deterministic behavior became progressively challenging as complexity grew.

To address these limitations, the architecture was redesigned to a bus-oriented model using the CAN bus and the **Cyphal** protocol. The CAN bus provides robustness, fault tolerance, and real-time capabilities, while Cyphal ensures deterministic data exchange and a structured, strongly typed message interface.

For more information about Cyphal, see: <https://opencyphal.org/>

Chapter 2

Cyphal Messages and Build System

2.1 Overview

All Cyphal message definitions in this project are generated automatically during the build process and placed in the `build` directory. Custom Python scripts integrated with CMake handle the generation. This is done similar to previous project with mavlink, to facilitate user to manipulate with protocol.

2.2 MAVLink to DSDL Conversion

The first script:

```
src/libs/libcanard/mavlink_to_dsd1.py
```

converts MAVLink messages from the `common/lisum` dialect to **DSDL (Data Structure Description Language)**. DSDL is used in Cyphal for strongly typed messages and services.

Generated DSDL files are stored in:

```
build/cyphal/cyphal/messages/mavlink/messages_cyphal_uavcan
```

This DSDL files are root for all `.h` `.c` files. With CMake we can generate it all.

For more information about DSDL, see: https://github.com/OpenCyphal/public_regulated_data_types

2.3 Official Cyphal Data Types

The project also includes original Cyphal-regulated DSDL files in:

`messages/cyphal`

These include both `reg` and `uavcan` namespaces, sourced from the official Cyphal repository. This is basic files for cyphal.

2.4 Automatic Code Generation

After DSDL sources are ready, the script:

`src/middleware/cyphal/gen_publishers.py`

generates corresponding `.c` and `.h` files. These provide:

- Strongly typed data structures
- Encoding/decoding functions
- API wrappers for message transmission and reception

In later chapter we will describe more about API, structures, encoding/decoding process.

Now generated files are stored in:

`build/cyphal/utility`

This entire process is fully integrated into the CMake build system, ensuring that message generation and source file creation are automated during compilation.

2.5 Figures: Memory Pools

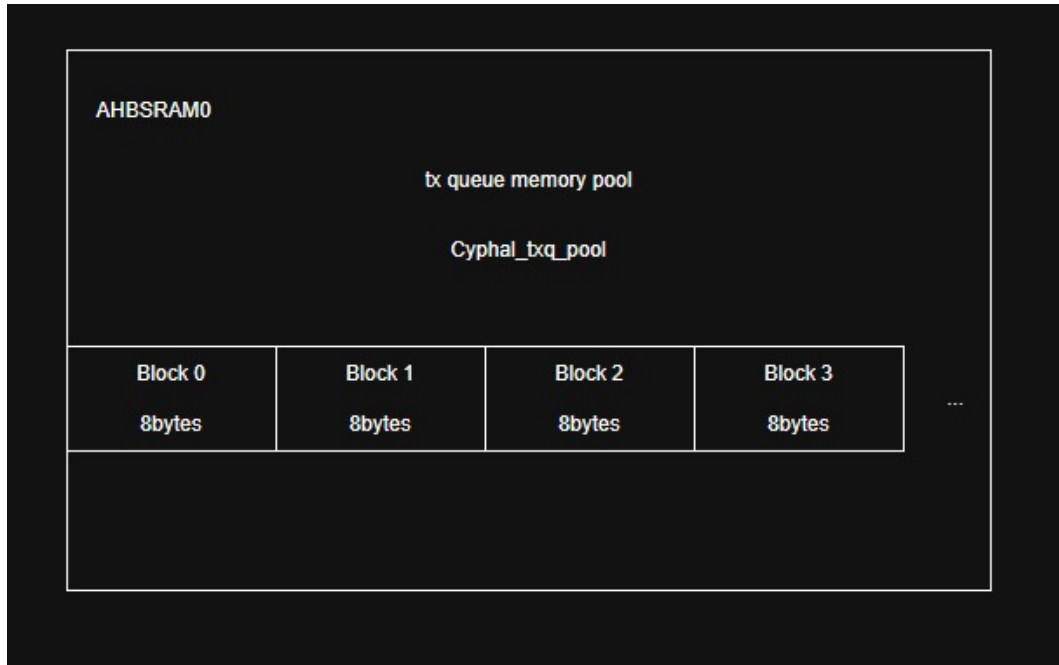


Figure 2.1: Transmit Queue (TXQ) Pool.

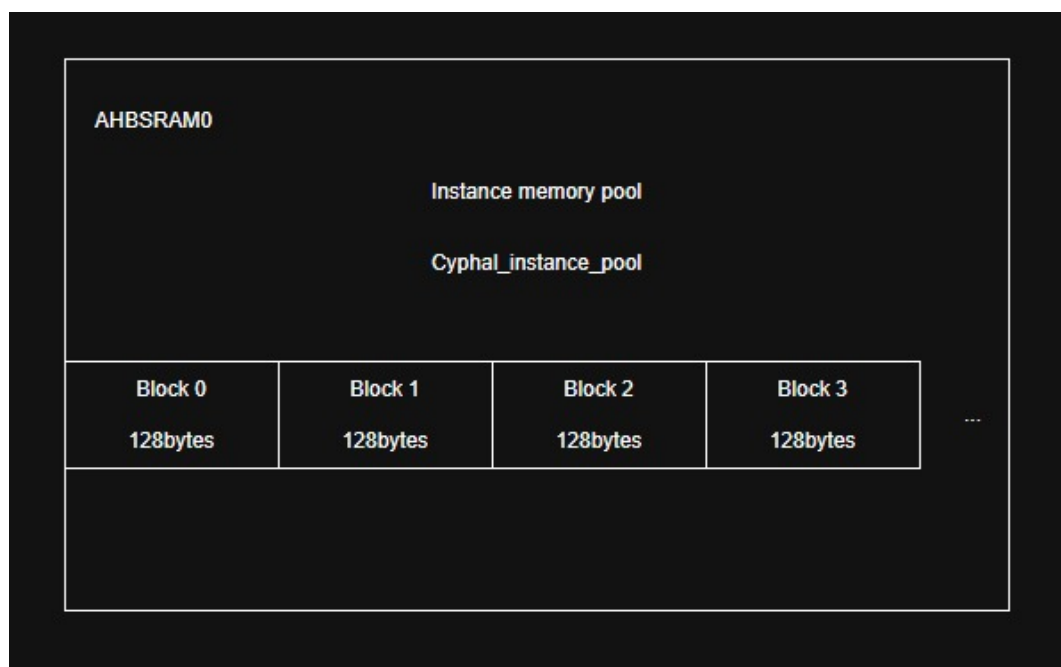


Figure 2.2: Libcanard Instance Pool.

Chapter 3

Libcanard

3.1 Overview

Libcanard is a lightweight library for implementing Cyphal on embedded systems. Features include:

- Deterministic message serialization/deserialization
- Node-ID filtering
- Dynamic memory management within a fixed pool
- Minimal CPU and memory footprint

The **Libcanard** library (<https://github.com/OpenCyphal/libcanard>) was used to implement this functionality, complemented by a custom middleware layer that simplifies its usage.

3.2 Memory Architecture

A statically allocated memory region (instance pool) is used as Libcanard's internal heap:

```
static uint8_t cyphal_instance_pool[1024];
```

Benefits:

- Deterministic allocation without fragmentation
- Complete control over memory usage
- Storage for all internal transfer/session structures

3.3 Transmit Queue (TXQ)

TXQ stores outgoing CAN frames with proper priority:

```
static CanardTxQueue txq;  
canardTxQueueInit(&txq, 100, 8);
```

Features:

- Correct priority ordering
- Buffering of multi-frame transfers
- Deterministic scheduling according to Cyphal/CAN rules

3.4 Initialization Flow

3.5 Initialization of Cyphal Components

The initialization of the Cyphal communication stack in our system follows three main steps: the memory pool, the main Canard instance, and the transmit queue.

3.5.1 Instance Pool

First, we initialize the `cyphal_pool`, a statically allocated memory region from the microcontroller’s RAM. From the perspective of the **Libcanard** library, this pool functions as a deterministic heap for all internal operations and data structures:

- Subscription states
- Transfer sessions
- Multi-frame transfer metadata
- Temporary buffers

Static allocation ensures predictable memory usage and avoids runtime fragmentation.

3.5.2 Main Canard Instance

Next, we initialize the `Canard` structure, which represents the core Cyphal context. This structure contains:

- References to user-defined data
- Node identifiers
- Pointers to allocated memory pools
- Subscription information

Each device in the system is assigned a unique `canard.node_id`. In our setup, these include nodes such as:

- `GW_SKY`
- `GW_GND`

- BLACK_BOX
- INS
- ACTUATORS

The node ID enables message filtering, ensuring that messages are only sent to intended recipients on the CAN bus.

3.5.3 Transmit Queue (TXQ)

Finally, we initialize the transmit queue (`tx_queue`), which handles outgoing messages. The queue:

- Fragments large messages into smaller CAN frames
- Buffers frames for deterministic transmission
- Interfaces with the hardware abstraction layer (HAL) to physically send CAN frames

This three-step initialization guarantees that the Cyphal stack is fully deterministic, memory-safe, and ready for real-time operation on embedded hardware.

Cyphal Initialization in `task_send`

```
/* ----- Cyphal Init ----- */

/* Initialize memory pools and Cyphal */
cyphal_pool_init();

/* Canard struct init */
canard = canardInit(cyphal_instance_memory);

/* Local node-ID (adjust as needed) */
canard.node_id = CYPHAL_BLACK_BOX_ID;

/* TX queue for one CAN interface */
tx_queue = canardTxInit(512,
                        CANARD_MTU_CAN_CLASSIC,
                        cyphal_txq_memory);
```

3.6 Subscription to messages

To efficiently manage incoming messages, we first define a structured array that allows iteration through all subscriptions using a `for` loop. This acts as a software filter, enabling the system to process only the messages of interest.

The subscription structure is defined as:

Subscription Structure Definition

```
typedef struct cyphal_subscription_messages_t {
    const enum CanardTransferKind kind;
    const CanardPortID port_id;
    const size_t serialization_buffer_size;
    struct CanardRxSubscription* subscription;
} cyphal_subscription_messages_t;
```

Each subscription corresponds to a specific subject-ID and includes metadata necessary for message parsing and buffer allocation. For example, the BlackBox node subscribes to the following messages:

BlackBox Node Subscriptions

```
cyphal_subscription_messages_t
cyphal_subscriptions[CYPHAL_SUBSCRIPTION_MESSAGES_COUNT_BLACKBOX] =
{
    {.kind = CanardTransferKindMessage,
     .port_id = common_ScaledImu_1_0_FIXED_PORT_ID_,
     .serialization_buffer_size =
         common_ScaledImu_1_0_SERIALIZATION_BUFFER_SIZE_BYTES_,
     .subscription = &sub_common_scaled_imu_1_0},

    {.kind = CanardTransferKindMessage,
     .port_id = common_ScaledPressure_1_0_FIXED_PORT_ID_,
     .serialization_buffer_size =
         common_ScaledPressure_1_0_SERIALIZATION_BUFFER_SIZE_BYTES_,
     .subscription = &sub_common_scaled_pressure_1_0}
};
```

Once the array is defined, a simple loop registers all subscriptions with the main **Canard** instance:

For loop for subscription

```
for (uint8_t i = 0;
    i < CYPHAL_SUBSCRIPTION_MESSAGES_COUNT_BLACKBOX;
    i++)
{
    canardRxSubscribe(
        &canard,
        cyphal_subscriptions[i].kind,
        cyphal_subscriptions[i].port_id,
        cyphal_subscriptions[i].serialization_buffer_size,
        CANARD_DEFAULT_TRANSFER_ID_TIMEOUT_USEC,
        cyphal_subscriptions[i].subscription
    );
}
```

This approach ensures:

- Efficient subscription management through iteration
- Clear mapping between subject-IDs and internal buffers
- Simplified addition or removal of subscriptions

By using this structured and automated method, the firmware maintains a clean, scalable, and deterministic way to handle Cyphal messages.

3.7 Processing Incoming Frames

Incoming CAN frames are processed inside the main task loop using the `cyphal_process` function. This function reads messages from the HAL layer, parses them into Cyphal frames, validates them, and then passes them to the message handler. Memory used for payloads is deallocated after processing.

Processing Incoming Cyphal Messages

```
void cyphal_process(void)
{
    twai_message_t rx_msg;

    while (twai_receive(&rx_msg, 0) == ESP_OK) {
        if (!(rx_msg.extd))
            continue;

        struct CanardFrame frame = {
            .extended_can_id = rx_msg.identifier,
            .payload          = {.size = rx_msg.data_length_code,
                                .data = rx_msg.data}};

        const CanardMicrosecond ts = esp_timer_get_time();
        struct CanardRxTransfer tr = {0};
        struct CanardRxSubscription* sub = NULL;

        int8_t acc = canardRxAccept(&canard,
                                    ts,
                                    &frame,
                                    0,
                                    &tr,
                                    &sub);

        if (acc > 0) {

            handle_cyphal_message(&tr);

            if (tr.payload.allocated_size > 0) {
                canard.memory.deallocate(
                    canard.memory.user_reference,
                    tr.payload.allocated_size,
                    tr.payload.data);
            }
        }
    }
}
```

****Explanation of the processing flow:****

1. Receive the next CAN frame from the hardware abstraction layer.
2. Ignore non-extended frames.
3. Convert the raw CAN message into a **CanardFrame**.
4. Call **canardRxAccept** to validate and assemble the Cyphal transfer.
5. If accepted, forward the transfer to **handle_cyphal_message**.
6. Deallocate dynamically allocated payload memory to avoid leaks.

This design ensures deterministic and safe processing of all incoming Cyphal messages on the embedded system.

The transfer handler decodes payloads and forwards them:

Cyphal Handler

```
static void handle_cyphal_transfer
(const struct CanardRxTransfer* tr)
{
    switch (tr->metadata.port_id) {
        case 251: /* NamedValueFloat */ break;
        case 246: /* ADS-B Vehicle */ break;
        case 301: /* AIS Vessel */ break;
    }
}
```

3.8 Publishing Messages

The node periodically publishes its heartbeat and mode status to the Cyphal network. The payload is serialized into a buffer and sent using a generated publisher function that handles fragmentation, priority, and insertion into the Libcanard TX queue.

Publishing Cyphal Heartbeat

```
uavcan_node_Heartbeat_1_0 node_mode_data = {0};
node_mode_data.value = current_node_mode;

uint8_t ptvt_buf[
uavcan_node_Mode_1_0_SERIALIZATION_BUFFER_SIZE_BYTES_];

size_t pt_sz = sizeof(ptvt_buf);

if (uavcan_node_Mode_1_0_serialize(
&node_mode_data, ptvt_buf, &pt_sz) >= 0)
{
    cyphal_publish_node_mode_subject(
        &canard,
        &tx_queue,
        CanardPriorityImmediate,
        uavcan_node_Mode_1_0_FIXED_PORT_ID_,
        ptvt_buf,
        pt_sz,
        &tid_node_mode_1_0,
        1000000U
    );
}
```

****Flow explanation:****

1. Fill the heartbeat/message structure with the current node mode.
2. Allocate a serialization buffer of appropriate size.
3. Serialize the data structure into the buffer.
4. If serialization succeeds, call the generated publisher function:
 - Pass the Canard instance and TX queue.

- Specify the Cyphal subject-ID and priority.
 - Provide the serialized payload buffer and its size.
 - Pass the transfer-ID and optional interval for periodic transmission.
5. The publisher automatically handles fragmentation and insertion into the TX queue.

This ensures deterministic and prioritized transmission of the node heartbeat on the Cyphal bus.

3.9 Draining the Transmit Queue

Once messages are packed into the Libcanard TX queue, they must be physically transmitted over the CAN bus. The function `cyphal_drain_tx_queue` polls the queue, sends frames using the default TX handler, and ensures deterministic transmission with proper priority arbitration.

Draining the Transmit Queue

```
int cyphal_drain_tx_queue(struct CanardInstance* ins,
struct CanardTxQueue* txq, uint32_t max_frames)
{
    if (!ins || !txq)
        return -CANARD_ERROR_INVALID_ARGUMENT;

    uint32_t sent = 0;

    while (sent < max_frames) {
        const CanardMicrosecond now = HAL_GetTimeUS();

        const int8_t r = canardTxPoll(txq,
                                      ins,
                                      now,
                                      NULL,
                                      cyphal_default_tx_handler,
                                      NULL,
                                      NULL);

        if (r > 0) {
            sent++;
        } else if (r == 0) {
            break;
        } else {
            return (int)r;
        }
    }

    return (int)sent;
}

int8_t cyphal_default_tx_handler(void* user_ref,
CanardMicrosecond deadline_usec, struct CanardMutableFrame* frame)
{
    (void)user_ref;
    (void)deadline_usec;
    twai_message_t msg;
    memset(&msg, 0, sizeof(msg));
    msg.identifier = frame->extended_can_id;
    msg.extd = 1; /* extended ID */
    msg.data_length_code = (uint8_t)frame->payload.size;
    memcpy(msg.data, frame->payload.data, msg.data_length_code);
    const esp_err_t st = twai_transmit(&msg, pdMS_TO_TICKS(200));
    if (st == ESP_OK) {
        return 1; /* transmitted */
    }

    return 0; /* retry later */
}
```

****Flow explanation:****

1. `cyphal_drain_tx_queue` is called with the Canard instance, TX queue, and max number of frames to send.
2. It repeatedly calls `canardTxPoll` to retrieve the next frame from the queue.
3. Each frame is sent via `cyphal_default_tx_handler`, which wraps the frame into a CAN message and transmits it through the HAL driver.
4. If a frame is successfully transmitted, the sent counter is incremented.
5. The loop stops when the maximum frames are sent or when no more frames are ready.
6. Deterministic priority arbitration is ensured by the TX queue and CAN protocol.

Chapter 4

Middleware for Cyphal Files

To simplify the use of the **Libcanard** library, we implemented a middleware layer. This middleware is contained in the files `cyphal_utility.c` and `cyphal_utility.h`.

The middleware serves several important purposes:

- Wraps low-level **Libcanard** functions into a higher-level API.
- Combines multiple library calls into single, user-friendly functions.
- Automates common Cyphal tasks such as:
 - Processing incoming messages (`cyphal_process`)
 - Draining the transmit queue and sending frames over the CAN bus (`cyphal_drain_tx_queue`)
 - Publishing messages with automatic fragmentation (`cyphal_publish_message`)
- Manages the entire Cyphal memory pool and internal buffers, ensuring deterministic memory allocation.
- Abstracts internal complexity, allowing developers to call a single function while the middleware handles the underlying library logic and combined processes.

Essentially, the middleware acts as an “automated” layer over **Libcanard**. From the user’s perspective, it functions like a high-level API: call a function, and the middleware takes care of message processing, memory management, queue handling, and message publication.

This approach improves maintainability, reduces repetitive code, and ensures consistent, deterministic behavior across the system.

Chapter 5

Overall Cyphal Communication Flow

1. Initialize memory pools, Canard instance, and TX queue
2. Subscribe to messages
3. Repeatedly process inbound CAN frames
4. Publish periodic messages
5. Push frames into TXQ
6. Drain TXQ and send via CAN
7. Forward decoded messages to MAVLink/application